



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Agentic AI for Video Creation

Course Code: C436

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	29 July 2026	Initial aligned release of the C436 deck, Learner Guide, Lesson Plan, and eight connected labs.	Assigned Course Trainer
2.0	16 September 2026	Rebuilt on Hermes Agent and MiniMax M3: three topics, FRAME-CUT prompt engineering, Remotion/Manim/Higgsfield/FFmpeg tool routing, custom brand-video skill, multi-agent Kanban review, private YouTube release, and paused cron publishing. Replaces the previous n8n-based labs.	Assigned Course Trainer

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Preparation.....	4
Topic 01 — Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering.....	5
Hermes Desktop Runtime and Shared CLI State.....	5
MiniMax M3 Provider, Trial Terms and Credential Isolation.....	7
The FRAME-CUT Video Prompt Contract.....	8
The Simple Video Agent Loop with Cost and Stop Rules.....	9
Lab 1 — Set Up Hermes Desktop and Connect MiniMax M3.....	10
Lab 2 — Prompt Hermes to Create a Simple Video.....	12
Lab 3 — Engineer Video Prompts with FRAME-CUT.....	14
Topic 02 — Video Tools, Hermes Skills and Custom Brand Production.....	15
Choosing Between a Tool and a Hermes Skill.....	16
Routing Remotion, Manim, Higgsfield and FFmpeg.....	17
Brand Tokens, Tone and the Custom Video Skill.....	18
The Technical Quality Gate, Provenance and Repair.....	19
Lab 4 — Install Video Tools and Hermes Skills.....	21
Lab 5 — Create a Custom Branded Video Skill.....	22
Topic 03 — Multi-Agent Kanban, YouTube Release and Scheduled Publishing.....	25
Agent Role Contracts and the Four Specialist Roles.....	25
Delegation Context, Parallel Work and the Durable Kanban.....	26
Human Approval, the YouTube Contract and Publishing Idempotency.....	27
Hermes Cron and Controlled Scheduled Release Operations.....	29
Lab 6 — Build the Multi-Agent Video Workflow.....	30
Lab 7 — Orchestrate Kanban Review and YouTube Upload.....	31
Lab 8 — Schedule Controlled Video Publishing with Hermes Cron.....	33
Integrated Workflow Wrap-Up.....	35
Next Steps.....	36
Glossary.....	36

Introduction

This Learner Guide accompanies Agentic AI for Video Creation (C436). It is a self-contained study text and practical reference for building an evidence-led video production system in Hermes Agent, from runtime setup and prompt engineering to custom video skills, multi-agent review, and controlled release.

The three topics and eight connected labs follow one Hermes-native journey. Work in order and retain each lab's evidence: later labs consume the runtime configuration, prompt contracts, skills, and approved masters created earlier.

Course Learning Outcomes

- LO1: Develop an editing strategy and work plan in Hermes Agent using MiniMax M3, structured video prompts and governed tool selection.
- LO2: Create and customise an evidence-backed video with Hermes skills, Remotion, Manim, Higgsfield-compatible requests and deterministic media checks.
- LO3: Orchestrate research, production, review and YouTube release agents through a durable Kanban board and controlled scheduled publishing.

Before You Start — Preparation

What you need

- A Windows or macOS laptop with a modern browser, administrator rights to install desktop software, and permission to create local folders.
- Hermes Desktop installed from the official page at <https://hermes-agent.nousresearch.com/desktop>.
- A MiniMax account and API key for the MiniMax-M3 model; trial, quota, and region terms are time-sensitive and must be confirmed live at sign-up.
- Python 3 on PATH for the supplied `verify.py` scripts and preview renderers.
- FFmpeg and FFprobe on PATH. On Windows, install with 'winget install --id Gyan.FFmpeg -e'; on macOS, use 'brew install ffmpeg'. Reopen the terminal and verify both commands.
- A Google account with a YouTube channel for the release labs; uploads stay private and public visibility is never required to complete a lab.
- The repository's labs folder, which contains eight lab packages with copy-ready prompts, synthetic data, starter files, evidence checklists, and verifiers.

Verify your setup

Confirm that Hermes launches, the configured provider is minimax, the model is MiniMax-M3, and the media tools return a version. Never place a real secret value in a prompt, lab file, screenshot, or public repository.

`hermes doctor`

```
ffmpeg -version
```

```
ffprobe -version
```

```
python3 --version
```

Conventions used in every lab

- Replace placeholders such as <RUN_ID> or <API_KEY> only in Hermes credential or configuration storage, never in a prompt or lab file.
- Use the supplied synthetic lab data. Do not add real customer, employee, creator, or account data.
- Keep YouTube visibility private. Public release requires an explicit trainer-supervised decision.
- Require a preview before any network call, paid generation, upload, or scheduling side effect.
- Run the lab's verify.py and retain the PASS output before starting the next lab.
- If a tool is unavailable, use the documented deterministic fallback and record the limitation rather than inventing a successful call.
- Confirm live trial, quota, and pricing terms in your own account at sign-up; treat any figure in this guide as an example, not a current offer.

Topic 01 — Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering

Course coverage: Day 1 morning and early afternoon | 3 labs.

Hermes Desktop runtime | MiniMax M3 provider contract | FRAME-CUT prompting | Structured shot plans

Key concepts

- Runtime boundary — The installed Hermes surface, its profile, configuration path and skill root that every run depends on.
- Provider contract — The named provider, base URL, model identifier and credential reference used for reasoning.
- FRAME-CUT — A prompt contract covering Format, Role, Action, Motion, Environment plus Continuity, Unwanted and Technical output.
- Shot plan — Strict JSON of timed shots whose durations total the approved target with no gaps or overlaps.

Hermes Desktop Runtime and Shared CLI State

Hermes Agent runs as a desktop application with a matching command-line surface. Both read the same profile, configuration path and skill directory, so a change made in one surface is visible to the other. Installation is completed from the official desktop page, after which a diagnostic command reports the health of each required

dependency. The runtime boundary is the set of facts that define where the agent executes: platform, installer source, version, active profile, configuration path and skill root.

An agent that cannot describe its own runtime cannot produce reproducible work. If the desktop and the terminal disagree about the active provider or skill directory, a lab that passes in one surface fails in the other for reasons that look like model behaviour. Recording the runtime boundary before any production work converts a class of confusing failures into a single observable check, and a failed dependency becomes a reason to stop rather than a problem to discover halfway through a render.

How it works

- Download the installer from the official Hermes desktop page and confirm the platform build.
- Complete setup, then run the diagnostic command and read every reported dependency.
- Open the desktop profile and note the configuration path and skill root.
- Open the terminal and compare provider, model and skill directory against the desktop.
- Stop and repair when a required diagnostic fails; do not continue into production work.

Worked example

- hermes doctor reports all required dependencies healthy on a supported platform.
- The desktop profile and the CLI session report the same provider, model and skill root.
- setup-evidence.json records the official source, version and diagnostic status with no credential value.

Decision guide

Use when	Avoid when
A new machine, a new learner account or a fresh profile is being prepared for production work.	A required diagnostic fails and the failure has not been repaired or recorded.
Results differ between the desktop and the terminal and the cause is not yet known.	The installer came from an unofficial mirror or the platform build is unverified.

Practitioner quality lens

- Failure signal: The same prompt behaves differently in the desktop and the terminal.
- Repair move: Name the active profile and compare provider, model and skill directory across both surfaces.
- Quality evidence: profile-check.json shows matching settings and the diagnostic reports no blocking failure.

MiniMax M3 Provider, Trial Terms and Credential Isolation

The provider contract names the service, base URL, model identifier and a reference to a stored credential. This course configures provider minimax with model MiniMax-M3 and verifies a tool-capable response before any production run. Trial availability, duration, quota and region terms are time-sensitive commercial terms rather than fixed course facts, so they are observed live at sign-up and recorded with a date. Credentials live in Hermes credential or configuration storage and are referenced by name; published evidence carries placeholders only.

A model handshake that is assumed rather than verified is the most common cause of a lab that fails much later, at the point of an expensive call. Verifying the returned model identifier proves that the configured model is the model actually answering. Separating the credential from the evidence keeps the lab package publishable: a secret that never enters a prompt, file, screenshot or repository cannot be leaked by one of them. Recording trial terms with a date and a fallback model means a changed offer becomes a planned decision rather than a blocked class.

How it works

- Create the account and an API key, then store the key in credential storage and reference it by name.
- Configure provider minimax with model MiniMax-M3 and confirm the returned model identifier.
- Open the live offer, read the current terms and record what was seen, the date and the expiry.
- Reserve a quota budget and nominate a fallback model before production work begins.
- Scan the lab package and confirm zero live tokens before publishing any evidence.

Worked example

- model-check.json records MiniMax-M3 and a successful tool-capable response without recording the key.
- trial-checklist.md contains the observed terms, the date seen and the learner's fallback decision.
- A secret scan over the lab package reports no live token pattern.

Decision guide

Use when	Avoid when
A provider or model is being configured for the first time, or a key has been rotated.	The returned model identifier does not match the configured model.
Quota, billing or region terms materially affect how much generation the run can afford.	A key would have to be pasted into a prompt, lab file, screenshot or repository to proceed.

Practitioner quality lens

- Failure signal: The session answers but reports a different model than the one configured.
- Repair move: Start a new session after selecting the model; an existing session can retain its original model.
- Quality evidence: budget-ledger.csv reconciles attempts, latency and cost against the approved ceiling.

The FRAME-CUT Video Prompt Contract

FRAME-CUT is the prompt contract used throughout this course. FRAME fixes Format and finish, Role and references, Action, Motion and Environment. CUT fixes Continuity tokens, Unwanted elements as explicit negative constraints, and the Technical output schema. A compliant prompt names one dominant subject action and one camera move per timed shot, states the palette and identity tokens that must persist, lists what must never appear, and requires strict JSON that can be validated before anything is generated.

A vague creative request such as 'make it cinematic' leaves the model to invent the channel, duration, subject, camera language and acceptance rule, and every regeneration invents them differently. A field-complete contract makes shots comparable across attempts, makes drift observable against named invariants, and makes the output checkable by a validator rather than by opinion. Because the contract is validated before generation, a defective plan costs a schema error instead of a paid render.

How it works

- Choose the channel, then freeze duration, aspect ratio and delivery codec before any shot work.
- Assign the role, attach the brief and register the source and asset identifiers that reasoning may use.
- Write one dominant action and one camera move for each timed shot.
- Lock continuity tokens and palette values, then list the must-avoid elements explicitly.
- Require strict JSON and fail the plan on gaps, overlaps, unknown fields or a duration mismatch.

Worked example

- A 15-second brief becomes a shot plan whose shot durations total exactly 15 seconds with no overlap.
- Each shot carries one action-motion pair, a continuity token and an explicit must-avoid list.

- prompt-contract.json passes the schema and duration checks before a renderer is called.

Decision guide

Use when	Avoid when
A creative intent must survive several regenerations or be handed to another agent or tool.	The request asks for imitation of a living artist or a named creator's identity.
The output will feed a deterministic renderer that requires named, typed fields.	The delivery format is still unsettled, so shot work would be rebuilt after the format changes.

Practitioner quality lens

- Failure signal: Shots look plausible individually but identity, palette or screen direction drifts between them.
- Repair move: Reuse the continuity token and compare keyframes against the explicit invariant list.
- Quality evidence: continuity-review.json records each deviation and the acceptance decision.

The Simple Video Agent Loop with Cost and Stop Rules

The agent loop receives a brief, plans shots, calls one tool, inspects the returned artifact and then stops or escalates. Its controls are numeric: an estimated cost, a maximum number of attempts, a timeout and a recorded stop reason. A run ledger links the run identifier, the plan hash, each tool call and each result so that the run can be reconstructed after the fact. The default posture is one render attempt, one review pass and an explicit repair decision rather than open-ended retrying.

Generation costs money and time, and an unbounded loop spends both without converging. Capping attempts converts a runaway loop into a decision that reaches a named person with the evidence already assembled. Recording the stop reason matters as much as recording success: a run that stopped because a dependency failed is a different operational fact from one that stopped because the budget ceiling was reached, and the two require different repairs.

How it works

- Estimate the number of calls and the cost before the first paid action.
- Set the maximum attempts, the timeout and the budget ceiling as explicit numbers.
- Run the loop: receive the brief, plan the shots, call one tool, inspect the result.
- Stop at the cap and record the stop reason rather than attempting another generation.
- Require a human decision before any additional paid generation is authorised.

Worked example

- run-ledger.json contains the plan, the tool call, the result reference and the stop reason.
- A 15-second preview is produced in one render attempt and probed before it is accepted.
- A second paid attempt is not started automatically; it is escalated with the evidence attached.

Decision guide

Use when	Avoid when
Work involves paid generation, external calls or any action with a real cost.	Attempts, timeouts and budget have not been given numeric limits.
A run must be explainable later to a reviewer who was not present when it executed.	The loop can start another paid generation without a recorded human decision.

Practitioner quality lens

- Failure signal: The loop keeps regenerating without reaching a defined finish state.
- Repair move: Cap attempts, add an explicit finish condition and route the remainder to a named owner.
- Quality evidence: The ledger shows one committed result per run identifier and a recorded stop reason.

Lab 1 — Set Up Hermes Desktop and Connect MiniMax M3

Learning outcome: LO1: configure the Hermes runtime and prove a tool-capable MiniMax M3 handshake without exposing a credential.

Goal: Establish the verified runtime boundary that every later C436 lab depends on.

You will install Hermes Desktop from the official source, configure the MiniMax provider for MiniMax-M3, run the diagnostic command, compare the desktop and CLI state, and record the live trial terms you actually see. Every piece of published evidence carries placeholders instead of secrets.

What you'll build

A setup-evidence.json recording the official source, version and diagnostic status, plus a redacted diagnostic screenshot. (Tools: Hermes Desktop, Hermes CLI, MiniMax M3, credential storage.)

Prerequisites

- A laptop with administrator rights to install desktop software.
- A MiniMax account able to create an API key.

- Open labs/lab-01-setup-hermes-and-connect-minimax-m3/ as the current Hermes project.

Step-by-step

1. Download official installer
2. Complete Hermes setup
3. Configure MiniMax-M3
4. Run hermes doctor
5. Verify model response
6. Redact evidence
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run python3 verify.py and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 01. Hermes launches, the configured provider is minimax, the model is MiniMax-M3, hermes doctor reports no blocking failure, and the published evidence contains placeholders only.

Checkpoint and rejoin point

The verified runtime boundary is recorded in setup-evidence.json. Every later lab assumes this provider, model and skill root.

Troubleshooting

If this happens	Fix
Hermes reports a different model than the one configured.	Start a new session after selecting MiniMax-M3; an existing session can retain its original model.
hermes doctor reports a failed dependency.	Repair the dependency before continuing; a failed diagnostic is a reason to stop, not a warning to pass.
The live trial terms differ from the course note.	Record the current live account terms with the date and use the fallback path; the offer is time-sensitive.

Challenge

Write the runtime boundary as a single JSON object and prove the desktop and CLI both report the same values for every field in it.

Reflection

Which runtime fact, if it silently changed between sessions, would be hardest to diagnose from the model's behaviour alone?

Note: The complete lab and its support-file references are in labs/lab-01-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Lab 2 — Prompt Hermes to Create a Simple Video

Learning outcome: LO1: convert a bounded brief into a validated shot plan and a deterministically rendered preview.

Goal: Produce a first end-to-end video whose plan and output are both machine-checked.

You will use a copy-ready Hermes prompt to turn a supplied 15-second brief into a strict-JSON shot plan, validate that the plan totals the target duration with no gaps or overlaps, run the deterministic preview renderer and probe the resulting MP4.

What you'll build

A simple-video.mp4 with its shot-plan.json and ffprobe.json evidence. (Tools: Hermes Desktop, MiniMax M3, Python preview renderer, FFprobe.)

Prerequisites

- Complete Lab 1 so the provider and model are verified.
- Open labs/lab-02-prompt-hermes-to-create-a-simple-video/ as the current Hermes project.
- Confirm data/video-brief.json and starter/render_preview.py are present.

Step-by-step

1. Open project folder
2. Submit bounded prompt
3. Validate shot plan
4. Run preview renderer
5. Probe MP4

6. Record evidence
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run `python3 verify.py` and retain the PASS output with the requested evidence.

Test it

`verify.py` must print PASS Lab 02. The shot plan is valid JSON, totals 15 seconds, uses only supplied assets, and the generated MP4 passes the dimensions, codec and duration checks.

Checkpoint and rejoin point

`shot-plan.json` and `ffprobe.json` establish the plan-then-probe pattern reused by every later production lab.

Troubleshooting

If this happens	Fix
The shot durations do not total the target.	Fix the plan before rendering; a duration mismatch is a schema failure, not a rendering problem.
The renderer cannot find an asset.	Check that the plan references only the supplied assets by their exact names.
A tool is missing.	Use the documented deterministic fallback and record the limitation instead of inventing a successful call.

Challenge

Introduce a deliberate one-second overlap between two shots and prove the validator rejects the plan before any render is attempted.

Reflection

Why is it cheaper to fail on the shot plan than on the rendered output, and what does that imply about where checks belong?

Note: The complete lab and its support-file references are in `labs/lab-02-*.md`. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Lab 3 — Engineer Video Prompts with FRAME-CUT

Learning outcome: LO1: apply the FRAME-CUT contract to make creative intent portable, comparable and checkable.

Goal: Turn a vague creative request into field-complete, scorable shot prompts.

You will diagnose a deliberately vague prompt, complete every FRAME field and every CUT field, generate strict shot JSON, score the result against a deterministic rubric and repair the single highest-severity defect.

What you'll build

A prompt-pack.json with its prompt-score.csv rubric evidence. (Tools: Hermes Desktop, MiniMax M3, FRAME-CUT template, scoring rubric.)

Prerequisites

- Complete Lab 2 so the plan-and-probe pattern is familiar.
- Open labs/lab-03-engineer-video-prompts-with-frame-cut/ as the current Hermes project.
- Review starter/frame-cut-template.md and data/prompt-cases.csv.

Step-by-step

1. Diagnose vague prompt
2. Complete FRAME fields
3. Complete CUT fields
4. Generate shot JSON
5. Score rubric
6. Repair one defect
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run python3 verify.py and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 03. Every shot carries all FRAME-CUT fields, one dominant action, one camera move, explicit continuity and negative constraints, and a measurable output check.

Checkpoint and rejoin point

prompt-pack.json is the reusable prompt contract that the tool-routing and brand-skill labs consume.

Troubleshooting

If this happens	Fix
A shot contains two competing camera moves.	Split it into two timed shots; one action-motion pair per shot keeps the result comparable.
Identity or palette drifts between shots.	Reuse the continuity token and compare keyframes against the explicit invariant list.
The rubric score is high but the output still looks wrong.	Check that the rubric measures the defect you observed; add the missing check rather than overriding the score.

Challenge

Hand your prompt pack to another learner and have them generate from it without discussion; every difference reveals a field still left implicit.

Reflection

Which FRAME-CUT field did you most want to leave blank, and what would the model have invented in its place?

Note: The complete lab and its support-file references are in labs/lab-03-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Topic 02 — Video Tools, Hermes Skills and Custom Brand Production

Course coverage: Day 1 late afternoon and Day 2 morning | 2 labs.

Key concepts

- Tool versus skill — A tool performs precise integrated execution; a skill packages repeatable instructions plus scripts.
- Skill anatomy — A directory holding SKILL.md with frontmatter, a procedure, bundled scripts and verification commands.
- Brand token — A stored, editable value for logo, palette, typography or motion referenced by the video skill.
- Quality gate — A fail-closed technical check on container, streams, dimensions, frame rate, duration and captions.

Choosing Between a Tool and a Hermes Skill

A tool is the right choice when a capability needs precise, integrated execution with its own authentication and error handling. A skill is the right choice when a procedure must be repeated reliably: it bundles instructions, scripts and templates in a directory containing SKILL.md, whose frontmatter declares the name, description and version. Hermes discovers skills by searching the catalogue, then loads the selected skill and only the resources it references, so context stays proportionate to the task.

Wrapping everything as a skill produces brittle prose where an integration was needed; wrapping everything as a tool produces unrepeatable one-off calls where a documented procedure was needed. Deciding explicitly, and recording why, keeps the production stack legible to the next person. Progressive loading matters for the same reason: a skill that pulls its entire reference tree into context on every invocation crowds out the working material the task actually needs.

How it works

- Identify the capability and its authentication mode, then decide tool or skill and name an owner.
- Create the skill directory and write frontmatter that describes the exact trigger.
- Add the procedure, bundle the scripts and include deterministic verification commands.
- Search the catalogue first, then view the selected skill and load only referenced resources.
- Test invocation and confirm the expected artifacts are produced.

Worked example

- decision-record.md states why each capability was implemented as a tool or as a skill.
- SKILL.md and a test transcript together show correct activation from the intended trigger.
- skill-usage.json records the selected skill and the resources that were loaded.

Decision guide

Use when	Avoid when
A procedure will be repeated across runs, learners or courses and must behave the same way each time.	A single ad-hoc call would do and no repetition is expected.
The capability needs bundled scripts and templates alongside its instructions.	The capability requires integrated authentication and precise error handling better served by a tool.

Practitioner quality lens

- Failure signal: A skill is not discovered when its trigger phrase is used.
- Repair move: Check the YAML frontmatter, the directory name and the SKILL.md filename, then restart discovery.
- Quality evidence: Repeated runs of the skill produce matching outputs from a clean start.

Routing Remotion, Manim, Higgsfield and FFmpeg

Each media tool has a distinct contract. Remotion renders code-driven compositions where duration in frames divided by frames per second gives the duration in seconds. Manim renders programmatic explanatory scenes whose labels and values are bound to an approved evidence model. A Higgsfield-compatible request selects a model, attaches references and writes a motion prompt, and is prepared as a preview before any account or quota is consumed. FFmpeg performs deterministic assembly: normalise every source, build the timeline, mix audio and encode the master.

Routing a task to a tool that cannot satisfy its contract is the most expensive category of error in a video pipeline, because the cost is usually discovered after generation. Deriving frames from duration rather than guessing prevents a composition that is silently the wrong length. Normalising sources before concatenation prevents the class of failures where clips join but the container reports inconsistent streams. Preparing a generation request as a preview keeps an unapproved asset or an exhausted quota from becoming a failed paid call.

How it works

- Classify the task, check the required modality and route it to a capable tool.
- For Remotion, derive duration frames from the approved duration and validate width, height, fps and codec after render.
- For Manim, bind labels and values to the approved evidence model and review the rendered frames.
- For a Higgsfield request, build a request preview first and call the service only with approved assets and quota.

- For FFmpeg, normalise every source before concatenation and probe the final container.

Worked example

- tool-routing.csv links every task to a capable model or tool with a recorded fallback.
- remotion-render.json and ffprobe.json agree with the composition contract.
- ffprobe.json confirms H.264 video, AAC audio, the expected dimensions, frame rate and duration.

Decision guide

Use when	Avoid when
A media task has a clear modality and an approved tool exists for it.	The required assets are unapproved or the account quota has not been confirmed.
The output must be verified deterministically rather than accepted on appearance.	A deterministic renderer would produce the result more reliably than a generative call.

Practitioner quality lens

- Failure signal: Clips concatenate but the final container reports inconsistent streams or the wrong duration.
- Repair move: Normalise every source to the same codec, dimensions and frame rate before rebuilding the timeline.
- Quality evidence: The probe of the final master matches the approved delivery specification field for field.

Brand Tokens, Tone and the Custom Video Skill

A brand token system stores the logo reference, palette values, font stack and motion rules as editable values that the video skill references rather than hard-codes. Tone and style controls describe the audience, the voice, the pacing in words per minute and an explicit list of stylistic exclusions. The custom brand-video skill combines these: it bundles scripts and templates, declares its inputs and outputs, and ships an acceptance test so that repeated runs can be compared rather than merely rerun.

Brand rules written into prose drift as soon as two people apply them. Stored as tokens and referenced from a skill, they become checkable: a frame either uses the approved palette value or it does not. Describing tone through attributes and exclusions, rather than by naming a living artist to imitate, keeps the output original and defensible. An acceptance test converts 'the skill works' into a repeatable comparison between a clean run and the expected artifacts.

How it works

- Extract the identity, then set the palette, typography and motion rules as stored tokens.
- Define the audience, voice and pacing, and write the style exclusions explicitly.
- Write the skill, bundle its scripts and bind its templates using relative or Hermes template paths.
- Render the video through the skill and review the frame samples against the checklist.
- Run the acceptance test from a clean start and version the resulting evidence.

Worked example

- brand-profile.yaml holds editable tokens and the frame samples pass the brand checklist.
- The custom skill is discoverable, produces an MP4 and passes the technical probe.
- A repeat run from a clean state produces matching outputs and the same evidence artifacts.

Decision guide

Use when	Avoid when
The same brand treatment must be applied across many videos or by several people.	The style brief asks for imitation of a living artist or a specific creator's identity.
Brand compliance needs to be demonstrated rather than asserted.	Brand values are still changing, so tokens would be rewritten immediately after use.

Practitioner quality lens

- Failure signal: Two runs of the same skill produce visibly different brand treatment.
- Repair move: Move the varying value into a stored token and reference it from the skill.
- Quality evidence: brand-review.json records the passed checks against the applicable brand tokens.

The Technical Quality Gate, Provenance and Repair

The quality gate probes the container, inspects representative frames, checks that captions are readable and then either passes or fails the master. It fails closed: a

missing stream, an out-of-bounds dimension or an unreadable caption blocks the master rather than producing a warning. Provenance runs alongside it — every asset is registered with its source, licence and a SHA-256 hash, so approval binds to an immutable version. A finding opens a repair, the repair renders a new version, and every affected check is rerun against that new version.

A video that looks correct in a player can still be technically invalid for the destination platform, and the failure surfaces at upload rather than in review. Deterministic probing catches that class of defect while it is still cheap to fix. Binding approval to a hash prevents the most damaging version-control error in a release pipeline: approving one master and shipping another. Re-testing the affected scope after a repair is what stops a fix in one place from silently breaking another.

How it works

- Probe the container and record the observed width, height, average frame rate and duration.
- Inspect frames and confirm captions are present and readable.
- Register each asset with its source, rights status and SHA-256 hash.
- Open a finding with its severity, assign the repair and render a new immutable version.
- Rerun every affected check against the new version and record the outcome.

Worked example

- technical-qc.json and the contact sheet show the observed values against the required bounds.
- asset-manifest.csv verifies rights status and hashes for every manifest row.
- repair-log.json links the finding, the change, the new hash and the re-test result.

Decision guide

Use when	Avoid when
A master is a candidate for release and must meet a declared delivery specification.	The delivery specification has not been agreed, so there is nothing to check against.
Approval will be given now and acted on later, so the approved artifact must be identifiable.	A finding would be closed without re-testing the checks it affects.

Practitioner quality lens

- Failure signal: A repair closes one finding and reopens another that was previously passing.
- Repair move: Write a new immutable version and rerun the full affected scope rather than the single check.
- Quality evidence: Closed findings outnumber reopened findings and each closure names its re-test.

Lab 4 — Install Video Tools and Hermes Skills

Learning outcome: LO2: build a governed tool registry and install learner-safe skills with recorded permissions and fallbacks.

Goal: Establish which capability is a tool, which is a skill, and what each is permitted to do.

You will inventory the local media tools, search the Hermes skill catalogue, install or create skills for Remotion, Manim, Higgsfield request preparation and FFmpeg verification, run a smoke test on each, and record the authentication mode, side effects and fallback for every entry in the registry.

What you'll build

A tool-routing.json registry with skill-smoke-test.json evidence. (Tools: Hermes skills, Remotion, Manim, Higgsfield request preview, FFmpeg.)

Prerequisites

- Complete Lab 3 so a validated prompt pack exists to route.
- Open labs/lab-04-install-video-tools-and-skills/ as the current Hermes project.
- Confirm FFmpeg and FFprobe return a version on PATH.

Step-by-step

1. Inventory local tools
2. Search Hermes skills
3. Install or create skills
4. Run smoke tests
5. Record permissions
6. Choose fallback
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run python3 verify.py and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 04. The registry records capability, installation state, auth mode, side effects and fallback, and every supplied SKILL.md passes its frontmatter and path checks.

Checkpoint and rejoin point

tool-routing.json governs which tool or skill each later production task is permitted to call.

Troubleshooting

If this happens	Fix
A skill is not discovered when its trigger is used.	Check the YAML frontmatter, the directory name and the SKILL.md filename, then restart skill discovery.
A smoke test loads far more context than the task needs.	Search first and load only the resources the selected skill actually references.
A tool is unavailable on this machine.	Record the limitation and route the task to the documented deterministic fallback.

Challenge

Add one capability to the registry that you deliberately implement as a tool rather than a skill, and record the reasoning in the decision record.

Reflection

Which capability was genuinely ambiguous between tool and skill, and which requirement finally decided it?

Note: The complete lab and its support-file references are in labs/lab-04-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Lab 5 — Create a Custom Branded Video Skill

Learning outcome: LO2: package brand, tone and style rules into a reusable skill and verify the rendered output.

Goal: Make brand compliance repeatable and demonstrable rather than asserted.

You will approve a brand profile, write SKILL.md for a custom brand-video skill, bind its templates using relative or Hermes template paths, render a custom video through the skill, review the frame samples against the brand checklist and version the resulting evidence.

What you'll build

A custom-video.mp4 with brand-review.json and render-evidence.json. (Tools: Hermes skills, brand token profile, Remotion or FFmpeg renderer, FFprobe.)

Prerequisites

- Complete Lab 4 so the tool registry and skills are installed.
- Open labs/lab-05-create-custom-branded-video-skill/ as the current Hermes project.
- Review data/brand-profile.yaml before editing any token.

Step-by-step

1. Approve brand profile
2. Create SKILL.md
3. Bind templates
4. Render video
5. Review frames
6. Version evidence
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run python3 verify.py and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 05. The custom skill is discoverable, uses relative or Hermes template paths, creates an MP4, passes the technical probe and meets every required brand token.

Checkpoint and rejoin point

The custom brand-video skill and its approved master are the production inputs for the multi-agent and release labs.

Troubleshooting

If this happens	Fix
Two runs produce visibly different brand treatment.	Move the varying value into a stored token and reference it from the skill.
The skill works interactively but fails from a clean start.	Replace any absolute or session-dependent path with a relative or Hermes template path.
The render looks correct but the probe fails.	Fix the delivery properties; a master that fails the gate is not a candidate for release.

Challenge

Change one palette token and prove that the rendered frames and the brand review both reflect the change without any edit to the skill's procedure.

Reflection

Which brand rule was hardest to express as a checkable token rather than as prose, and how did you make it measurable?

Note: The complete lab and its support-file references are in labs/lab-05-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Topic 03 — Multi-Agent Kanban, YouTube Release and Scheduled Publishing

Course coverage: Day 2 afternoon | 3 labs.

Agent role contracts | Delegation and parallel work | Durable Kanban | Approval hash and cron release

Key concepts

- Role contract — One agent, one owned deliverable, a bounded tool list and one measurable completion rule.
- Delegation context — The exact goal, paths, inputs, constraints and acceptance tests handed to a child agent.
- Durable Kanban — Task state and dependencies that survive restarts and deliberate human pauses.
- Approval hash — A named decision bound to the immutable hash of the release package it authorises.

Agent Role Contracts and the Four Specialist Roles

Each agent receives a contract: a goal, a bounded list of allowed tools, one named deliverable and an explicit completion rule. This course uses four roles. The research agent reads the brief, retrieves sources, scores evidence and hands off claims with retrievable URLs and stated limitations. The video production agent consumes approved claims, creates the shot plan, invokes the brand skill and returns a master with its hash. The independent review agent inspects evidence against a rubric and creates timecoded findings. The upload agent verifies approval, builds metadata and uploads privately.

Roles without contracts drift into each other until no agent owns the outcome and two agents rewrite the same artifact. One deliverable per agent makes ownership unambiguous and makes a stalled pipeline diagnosable — the incomplete deliverable identifies the responsible role. The reviewer is deliberately denied approval authority: a reviewer that can approve its own findings provides no independent check, because the same assumptions that produced the work would clear it.

How it works

- Define the goal, limit the tools, name the deliverable and set the completion rule for each role.
- Require the research agent to produce retrievable URLs, bounded excerpts and explicit limitations.
- Give the production agent immutable research input and require the custom brand-video skill.
- Run the reviewer in an independent context and withhold approval authority from that role.

- Block the upload agent until every parent deliverable and the current approval hash pass.

Worked example

- agent-contracts.yaml validates all four specialist roles against the required fields.
- research-handoff.json contains approved sources and claim identifiers.
- review-handoff.json contains timecoded findings and a QA status but no approval decision.

Decision guide

Use when	Avoid when
Distinct roles have genuinely different tools, inputs or review criteria.	One bounded prompt and one tool would complete the task more reliably.
An independent check is needed before a consequential external action.	Agents would share unrestricted credentials or be able to overwrite accepted artifacts.

Practitioner quality lens

- Failure signal: Two agents disagree about which artifact is current.
- Repair move: Give each deliverable one owning role and one immutable version identifier.
- Quality evidence: Every factual claim in the output traces to a sourced claim identifier.

Delegation Context, Parallel Work and the Durable Kanban

A delegation context packages the goal, the required context, the exact paths and the acceptance tests before a child agent is dispatched. Independent tasks may run in parallel, but the join happens only on explicit verified artifacts rather than on elapsed time. The Hermes Kanban makes this durable: tasks carry an assignee, a status and parent identifiers, and the board survives restarts and deliberate human pauses, which is what allows a production run to span a break, a review or an overnight wait.

A child agent that receives a goal without paths, inputs and tests will produce something plausible that does not fit the pipeline, and the mismatch is usually discovered at the join. Parallelising dependent work is worse: it appears faster while silently skipping a dependency. A durable board is what separates a multi-agent workflow from a long single conversation — work that must survive a restart cannot live only in a session, and a paused human review is a normal state rather than a failure.

How it works

- Package the goal, context, exact paths and acceptance criteria into the delegation context.

- Dispatch the child agent, receive its summary and verify the artifact it claims to have produced.
- Identify genuinely independent tasks and dispatch those as a batch.
- Create the board tasks, assign profiles and link every dependency before running the dispatcher.
- Join only after all required parents are complete and verified.

Worked example

- delegation-log.json records the prompt, the agent and the verified result.
- join-ledger.json shows every required parent complete before the join proceeded.
- kanban-export.json shows the full dependency chain with no missing dependency.

Decision guide

Use when	Avoid when
Work crosses several agents and must survive restarts or a human pause.	The tasks are dependent, so parallel dispatch would skip a dependency.
Independent tasks exist and the join can be defined on verified artifacts.	The work fits one session and gains nothing from durable state.

Practitioner quality lens

- Failure signal: A task repeats the same failure and the pipeline retries it indefinitely.
- Repair move: Cap attempts, classify the failure and block repeated failures for human intervention.
- Quality evidence: Task history shows checkpoints, the review and the final evidence for each task.

Human Approval, the YouTube Contract and Publishing Idempotency

Before release the package is frozen and hashed, and a named reviewer approves that specific hash. Immediately before upload the current master hash is compared against the approved hash; a mismatch blocks the upload. The YouTube videos.insert contract requires OAuth authorisation, a snippet, a status block carrying the privacy setting and, where applicable, the synthetic-media disclosure. Uploads default to private. An idempotency key is created before the attempt and reconciled after it, so one approved package yields at most one committed video identifier.

Publishing is irreversible in a way that almost nothing earlier in the pipeline is. Approving a hash rather than a filename closes the gap where an artifact changes between approval and upload. Defaulting to private means a mistake is recoverable by deleting a private video rather than by retracting a public one. Reconciling an idempotency key before a retry prevents the specific failure where a network timeout on a successful upload produces a second copy on the channel.

How it works

- Freeze the release package, compute its hash and request approval against that hash.
- Recheck the current master hash immediately before upload and block on any mismatch.
- Authorise OAuth, confirm the channel identity, then set the snippet and the status block.
- Default privacy to private and include the synthetic-media disclosure where applicable.
- Create the publish key, check any prior result, then upload and commit the returned identifier.

Worked example

- approval-ledger.json proves the reviewer, the time, the decision and the approved hash.
- request-preview.json and the private upload receipt match the approved package.
- publication-ledger.csv contains exactly one committed video identifier per idempotency key.

Decision guide

Use when	Avoid when
An external, irreversible action is about to be taken on an approved artifact.	The approved hash does not match the current master hash.
Retries are possible and duplicate posts would be damaging.	Channel identity, disclosure or rights are unresolved, or public visibility is proposed without supervision.

Practitioner quality lens

- Failure signal: A retry after a timeout produces a second video on the channel.
- Repair move: Reconcile the prior idempotency key and its committed video identifier before retrying.
- Quality evidence: The duplicate publish rate is zero across all recorded attempts.

Hermes Cron and Controlled Scheduled Release Operations

A cron job runs in a fresh session, so its prompt must be self-contained: every required path, gate and stop rule is written into the job, and the skills it needs are attached explicitly. The job is created paused. It is exercised with a dry run whose output is inspected, and the next scheduled run is confirmed before any cadence is enabled. Scheduled work in this course prepares the release and performs the private upload; public visibility remains a separate, explicitly approved decision.

A scheduled job inherits nothing from the session that created it, so a prompt that relies on conversational context will behave differently at three in the morning than it did during testing. Creating the job paused makes that difference observable before it matters. Restricting the schedule to preparation and private upload means the worst outcome of a scheduling defect is an unwanted private draft rather than an unapproved public post.

How it works

- Write a self-contained job prompt containing every required path, gate and stop rule.
- Attach the custom video skill and any other required skills explicitly.
- Create the schedule with an explicit timezone and leave the job paused.
- Trigger a dry run, inspect the output and confirm the next run time.
- Enable the cadence only after the evidence has been reviewed and the enablement approved.

Worked example

- cron-preview.json records the schedule, the next run and the paused state.
- operations-ledger.csv records each run, its outcome and the owner's decision.
- The dry run cannot publish an unapproved or duplicate video.

Decision guide

Use when	Avoid when
A release cadence is required and the preparation steps are already reliable.	The job prompt still depends on context from the session that created it.
The job can be made fully self-contained and exercised with a dry run first.	Enabling the cadence would allow a public post without a separate approval.

Practitioner quality lens

- Failure signal: A job that passed interactively behaves differently on its first scheduled run.
- Repair move: Move every implicit path, gate and stop rule into the job prompt and re-run the dry run.
- Quality evidence: The on-time success rate holds with zero unauthorised posts.

Lab 6 — Build the Multi-Agent Video Workflow

Learning outcome: LO3: define four bounded specialist roles and prove that upload stays blocked until every gate passes.

Goal: Give each stage one owner, one deliverable and one measurable completion rule.

You will define and simulate four isolated Hermes roles for research, video creation, independent review and approved YouTube upload. Each role receives a bounded tool list and a named deliverable, the reviewer runs in an independent context without approval authority, and the uploader stays blocked until all parent evidence and the current approval hash pass.

What you'll build

A multi-agent-plan.json with four verified handoff records. (Tools: Hermes delegation, agent contracts, handoff schema.)

Prerequisites

- Complete Lab 5 so an approved branded master and its hash exist.
- Open labs/lab-06-build-multi-agent-video-workflow/ as the current Hermes project.
- Review data/agent-contracts.yaml and data/handoff-schema.json.

Step-by-step

1. Define role contracts
2. Package context
3. Delegate research
4. Delegate production
5. Request review
6. Gate uploader
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run python3 verify.py and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 06. All roles have bounded tools and outputs, the reviewer is independent, and upload is blocked until all parent evidence and the current approval hash pass.

Checkpoint and rejoin point

The four validated role contracts are the agents the Kanban board dispatches in the next lab.

Troubleshooting

If this happens	Fix
Two agents rewrite the same artifact.	Give each deliverable one owning role and one immutable version identifier.
A child agent returns something plausible that does not fit the pipeline.	Add the exact paths, inputs and acceptance tests to the delegation context and redispach.
The reviewer clears its own findings.	Run the reviewer in an independent context; a reviewer with approval authority provides no independent check.

Challenge

Change the master after approval and prove the uploader blocks on the hash mismatch rather than uploading the newer file.

Reflection

Where did splitting a stage into its own agent genuinely improve control, and where did it only add a handoff?

Note: The complete lab and its support-file references are in labs/lab-06-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Lab 7 — Orchestrate Kanban Review and YouTube Upload

Learning outcome: LO3: run a durable dependency chain and bind a named approval to the exact release hash.

Goal: Make the release decision durable, independent and reconcilable.

You will create a Hermes Kanban dependency chain, assign profiles, link every parent, require review, approve the exact package hash and then prepare or execute a private YouTube upload under explicit human authorisation with an idempotency key.

What you'll build

A `kanban-export.json`, `approval-ledger.json` and a private-upload receipt or dry-run preview. (Tools: Hermes Kanban, approval ledger, YouTube Data API `videos.insert`.)

Prerequisites

- Complete Lab 6 so the four role contracts are validated.
- Open `labs/lab-07-orchestrate-kanban-review-and-youtube-upload/` as the current Hermes project.
- A Google account with a YouTube channel; uploads remain private throughout.

Step-by-step

1. Create board tasks
2. Assign profiles
3. Link dependencies
4. Request review
5. Approve exact hash
6. Upload private
7. Open this lab folder as the current project in Hermes Desktop.
8. Read `AI-PROMPTS.md`; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run `python3 verify.py` and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 07. The Kanban graph has no missing dependency, review is required, the request points to the approved master, privacy is private, and no credential is present in any artifact.

Checkpoint and rejoin point

approval-ledger.json and the private upload receipt are the release evidence the scheduled-publishing lab operates against.

Troubleshooting

If this happens	Fix
Review or upload is blocked.	Inspect the exact dependency status and compare the approved payload hash with the current master hash.
A retry after a timeout produces a second video.	Reconcile the prior idempotency key and its committed video identifier before retrying.
A task fails repeatedly.	Cap the attempts, classify the failure and block it for human intervention rather than retrying indefinitely.

Challenge

Remove one dependency link and prove the board lets a task start early, then restore it and show the join waiting on verified artifacts.

Reflection

Why does approving a hash rather than a filename close a gap that filename-based approval leaves open?

Note: The complete lab and its support-file references are in labs/lab-07-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Lab 8 — Schedule Controlled Video Publishing with Hermes Cron

Learning outcome: LO3: create a paused, self-contained scheduled job and enable a cadence only after evidence review.

Goal: Make scheduled release safe by default and observable before it runs unattended.

You will write a self-contained cron prompt containing every required path, gate and stop rule, attach the custom video skill explicitly, create the schedule paused with an explicit timezone, trigger a dry run, inspect the next run time and approve enablement only after reviewing the evidence.

What you'll build

A cron-preview.json with an operations-ledger.csv. (Tools: Hermes cron, attached skills, operations ledger.)

Prerequisites

- Complete Lab 7 so an approved release package and its ledger exist.
- Open labs/lab-08-schedule-controlled-video-publishing/ as the current Hermes project.
- Review starter/release-job-prompt.md and starter/cron-commands.md.

Step-by-step

1. Write self-contained job
2. Attach video skill
3. Create paused cron
4. Trigger dry run
5. Inspect next run
6. Approve enablement
7. Open this lab folder as the current project in Hermes Desktop.
8. Read AI-PROMPTS.md; replace only the named placeholders with supplied synthetic values.
9. Ask Hermes to inspect the local files before it proposes a plan.
10. Require a preview before any network, paid-generation, upload or scheduling side effect.
11. Run python3 verify.py and retain the PASS output with the requested evidence.

Test it

verify.py must print PASS Lab 08. The schedule and timezone are explicit, the prompt is self-contained, the custom skill is attached, the job begins paused, and the dry run cannot publish an unapproved or duplicate video.

Checkpoint and rejoin point

cron-preview.json and operations-ledger.csv complete the handover package for the whole C436 pipeline.

Troubleshooting

If this happens	Fix
A job that passed interactively behaves differently on its first scheduled run.	Move every implicit path, gate and stop rule into the job prompt; a cron job inherits no session context.
The attached skill is not found at run time.	Attach the skill explicitly to the job rather than relying on discovery from the creating session.
The dry run attempts a real publish.	Keep the job paused and confirm the release gate requires a separate approval for public visibility.

Challenge

Remove one path from the job prompt and show the scheduled run failing in a way the interactive run did not, then restore it.

Reflection

What is the worst outcome a defect in your schedule could now cause, and which control bounds it to that?

Note: The complete lab and its support-file references are in labs/lab-08-*.md. Use only supplied or authorised assets. Never paste a MiniMax key, OAuth token, or YouTube credential into a prompt, lab file, screenshot, or repository. YouTube examples default to private and scheduled publishing starts paused.

Integrated Workflow Wrap-Up

The completed C436 project is not merely a generated video. It is a controlled production system whose artifacts, decisions, and evidence can be reviewed, resumed, and improved.

Minimum handover package

- Setup evidence, profile check, and model check with no credential value recorded.

- FRAME-CUT prompt pack, validated shot plan, and the preview probe.
- Tool routing registry, installed skills, brand profile, and the custom video skill.
- Agent contracts, Kanban export, review findings, approval ledger, and the cron preview.

Operational rule

When a stage cannot prove that its required input is approved, it must stop or escalate. It must not silently invent the missing fact, permission, file, or decision.

Next Steps

- Rerun the eight labs from a clean profile and explain every recorded state transition.
- Replace one deterministic fallback with an approved live integration while preserving the same contract, limits, and evidence.
- Add three representative test jobs, including one missing-input case and one tool-failure case.
- Pilot with private outputs, review the repair and cost evidence, and enable a cadence only after the guardrails remain stable.
- Review the official Hermes, MiniMax, Remotion, Manim, and YouTube documentation before adapting any live API or interface shown in this guide.
- Recheck YouTube upload and privacy requirements at https://developers.google.com/youtube/v3/guides/uploading_a_video before any authorised public release.

Glossary

- **Agent loop** — The repeated observe, plan, act, inspect, decide, and record cycle.
- **Approval hash** — A named decision bound to the immutable hash of the package it authorises.
- **Asset manifest** — The authoritative inventory of assets with source, rights status, version, and checksum.
- **Brand token** — A stored, editable value for logo, palette, typography, or motion referenced by a skill.
- **Circuit breaker** — A cap on attempts that blocks a repeatedly failing task for human intervention.
- **Continuity token** — A reusable identifier that holds subject identity and palette stable across shots.
- **Cron job** — A scheduled Hermes task that runs in a fresh session from a self-contained prompt.
- **Delegation context** — The goal, context, exact paths, and acceptance tests handed to a child agent.

- **FRAME-CUT** — The prompt contract covering Format, Role, Action, Motion, Environment, Continuity, Unwanted, and Technical output.
- **Hermes Desktop** — The desktop application surface of Hermes Agent, sharing profile and configuration with the CLI.
- **Idempotency key** — A stable identifier used to prevent a retry from creating a duplicate external action.
- **Kanban** — A durable board of tasks with assignees, statuses, and parent dependencies that survives restarts.
- **MiniMax M3** — The reasoning and tool-orchestration model configured through the minimax provider.
- **Progressive loading** — Searching the skill catalogue first and loading only the resources a selected skill references.
- **Provenance** — Recorded information about an asset's source, licence, version, and checksum.
- **Quality gate** — A fail-closed technical check on container, streams, dimensions, frame rate, duration, and captions.
- **Shot plan** — Strict JSON of timed shots whose durations total the approved target with no gaps or overlaps.
- **Skill** — A directory containing SKILL.md, scripts, and templates that package a repeatable procedure.
- **Synthetic media disclosure** — The platform status field declaring that content was generated or materially altered.